

Simple Query Optimizer

A Standalone Database Optimization Engine

DBMSTeam 404

Roll Number	Name
23CS10054	Popuri Anirudh (c)
23CS10013	Chintada Yesheeth Sree Narayana
23CS10078	Vemuri Sai Lokesh
23CS10052	Payidiparthi Sri Surya Naga Hadwik (vc)
23CS10017	Dasari Veera Venkata Abhinav Teja

Department of Computer Science and Engineering

Course: Database Management Systems Lab

April 14, 2026

Contents

1	Objective and Abstract	2
1.1	Objective	2
1.2	Abstract	2
1.3	System Pipeline	2
2	Methodology	4
2.1	Architecture Overview	4
2.2	Development Phases	4
3	System Components	5
3.1	Lexer and Parser (<code>lexer.l</code> / <code>parser.y</code>)	5
3.2	Expression Trees (<code>expressions.hpp</code>)	5
3.3	AST Plan Nodes (<code>ast_nodes.hpp</code>)	6
3.4	System Catalog (<code>catalog.hpp</code>)	6
3.5	Semantic Analyzer (<code>semantic_analyzer.hpp</code>)	7
3.6	Rule-Based Optimizer — Predicate Pushdown (<code>optimizer.hpp</code>)	7
3.6.1	AND-Predicate Splitting	7
3.6.2	Single-Predicate Pushdown	8
3.7	Cost Model (<code>cost_model.hpp</code>)	8
3.7.1	Sequential Scan	8
3.7.2	Nested Loop Join (NLJ)	9
3.7.3	Hash Join	9
3.8	Cost-Based Optimizer — DP Join Enumeration (<code>cost_based_optimizer.hpp</code>)	9
3.8.1	Phase 1 — Tree Deconstruction	9
3.8.2	Phase 2 — Bitmask DP Enumeration	9
3.8.3	Phase 3 — Plan Reassembly	10
4	Summary of Results	11
5	Web Dashboard	11

Objective and Abstract

Objective

The goal of this project is to design and implement a **standalone, cost-based SQL query optimization engine** that:

- Parses raw SQL DML statements into a structured Abstract Syntax Tree (AST) using a Flex lexer and Bison parser.
- Validates the query semantically against a catalog of table and column definitions.
- Applies **Rule-Based Optimization (RBO)** to push filter predicates as close to the data source as possible (Predicate Pushdown).
- Applies **Cost-Based Optimization (CBO)** using a System R-inspired dynamic programming join enumerator to select the cheapest physical join ordering and algorithm.
- Exposes results through both a command-line interface and an interactive web dashboard driven by D3.js.

Abstract

Database query performance degrades sharply as query complexity grows. For a query joining N tables, the number of possible join orderings is $N!$, creating a combinatorial explosion that makes exhaustive search infeasible beyond $N \approx 6$ tables. This project addresses this problem by building a modular optimizer that combines algebraic rewriting rules with statistical cost modeling.

The system is structured as a three-stage pipeline: (i) a *lexer/parser* that translates raw SQL into a typed AST using a JSON bridge between Bison (C) and the C++17 backend; (ii) an *RBO* that applies predicate pushdown via structural tree rewrites; and (iii) a *CBO* that uses bitmask dynamic programming with Gosper's Hack subset enumeration and a PostgreSQL-inspired cost model to find the optimal bushy join tree. The system supports both a mock JSON catalog for rapid testing and a live PostgreSQL statistics backend. All three plan stages are visualized interactively via a Flask web server and a D3.js front-end.

System Pipeline

The following figure illustrates the complete data flow from a user's SQL query to the final optimized physical plan.

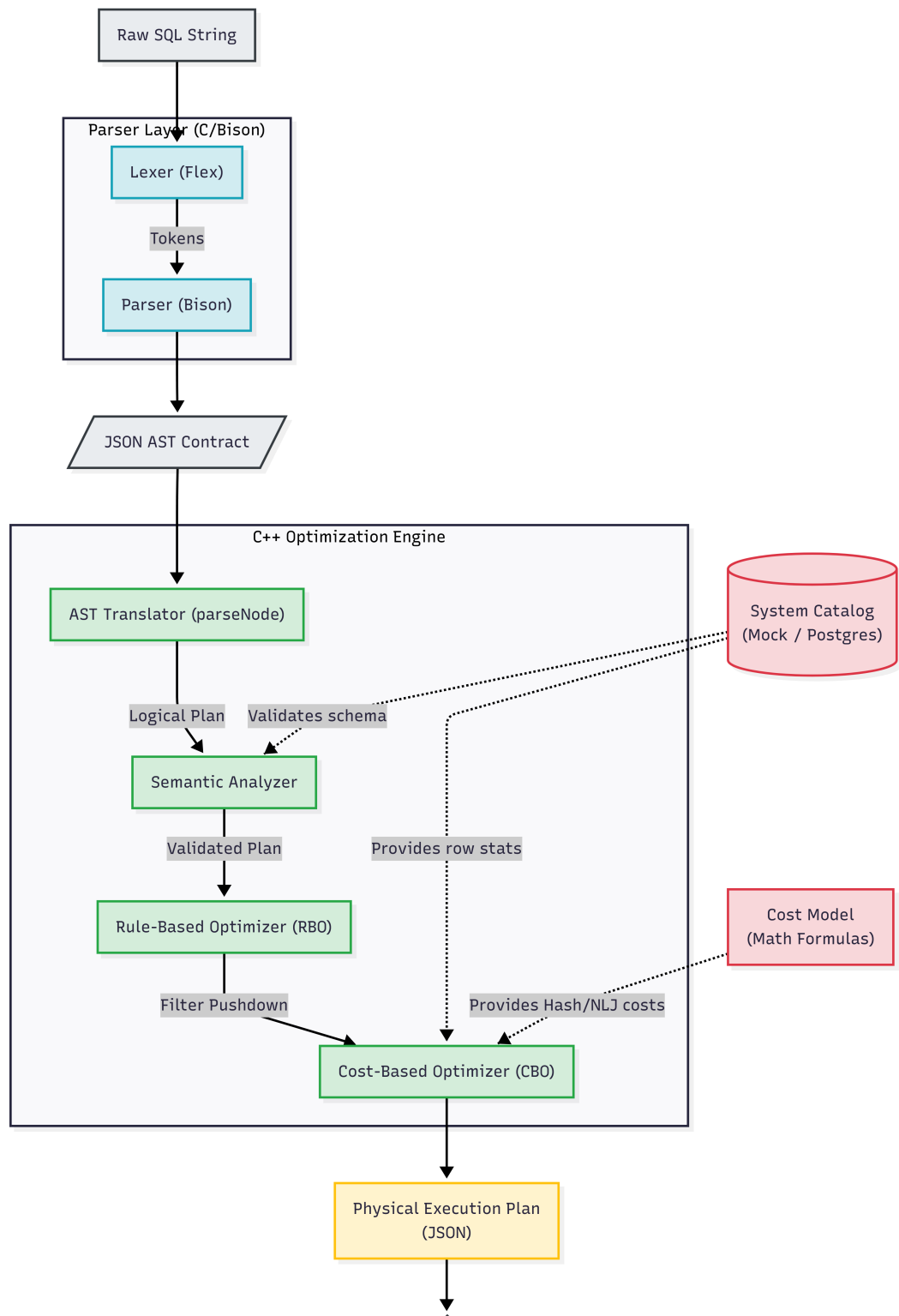


Figure 1: Data flow from SQL query to physical execution plan.

Methodology

Architecture Overview

The engine is implemented in **C++17** and organized into seven cooperating modules. Each module communicates through well-defined interfaces, keeping the components independently testable and replaceable.

Table 1: Module summary

Module	Responsibility
<code>lexer.l / parser.y</code>	Tokenize raw SQL; produce a JSON AST via a Flex+Bison pipeline
<code>expressions.hpp</code>	Recursive expression tree nodes for predicates and conditions
<code>ast_nodes.hpp</code>	Typed logical and physical plan nodes forming the query tree
<code>catalog.hpp</code>	ICatalog interface with MockCatalog (JSON) and PostgresCatalog backends
<code>semantic_analyzer.hpp</code>	Bottom-up tree walk validating tables, aliases, and column references
<code>optimizer.hpp</code>	RBO: predicate pushdown with AND-splitting
<code>cost_based_optimizer.hpp</code>	CBO: DP join enumeration with bitmask subset iteration
<code>cost_model.hpp</code>	Static cost formulas for sequential scan, index scan, Hash Join, NLJ
<code>main.cpp</code>	CLI orchestration; JSON snapshots; Flask/D3 integration

Development Phases

The project was implemented over five weeks in accordance with the original proposal:

1. **Week 1 — Representation and Ingestion:** Defined the `PlanNode` hierarchy, `Expression` trees, and the Flex/Bison pipeline. Established the JSON bridge between the C parser and the C++ backend.
2. **Week 2 — Statistics and Cost Modeling:** Built `MockCatalog` (JSON-driven) and `PostgresCatalog` (live `pg.class` queries). Implemented the five-constant cost model.
3. **Week 3 — Rule-Based Optimizer:** Implemented predicate pushdown with AND-predicate splitting. Integrated the semantic analyzer as a gating check before any optimization.
4. **Week 4 — Cost-Based Optimizer:** Implemented the bitmask DP enumerator, Gosper’s Hack subset iteration, cardinality estimation, and the `PRUNE_BUDGET` pruning mechanism.
5. **Week 5 — Integration and Testing:** Built the Flask web server, D3.js dashboard, and ran few end-to-end test queries covering single-table scans, multi-table DP joins, GROUP BY, LEFT JOIN, semantic errors, AND-filter splitting, and alias resolution.

System Components

Lexer and Parser (lexer.l / parser.y)

The front-end translation layer converts a raw SQL string into the structured JSON AST consumed by the C++ backend.

Lexer (Flex). `lexer.l` applies regular-expression rules to tokenize the input. Reserved keywords (`SELECT`, `FROM`, `JOIN`, `WHERE`, ...) are matched literally and returned as named tokens. Identifiers are matched by the pattern `[a-zA-Z_][a-zA-Z0-9_]*`, with the matched string copied into `yylval.str` via `strdup(yytext)`.

Parser (Bison). `parser.y` enforces the SQL grammar. Each production rule assembles the matching tokens into a JSON node. For example, a `WHERE` clause:

```
1 where_clause:
2     WHERE condition { $$ = $2; }
3   | /* empty */    { $$ = nullptr; }
4   ;
```

JSON Bridge. Because Bison is a C tool and the optimizer is C++17, direct instantiation of `unique_ptr<PlanNode>` inside the parser would require mixed-language memory management and is error-prone. Instead, every grammar action builds a **JSON object** using the `nlohmann/json` library:

```
1 condition AND condition {
2     $$ = new json({
3         {"expression_type", "LOGICAL_AND"},
4         {"left",    *$1},
5         {"right",   *$3}
6     });
7 }
```

The function `parse_sql_to_json()` returns the completed JSON tree to `main.cpp`, which then calls `parseNode()` to safely construct the typed C++ `PlanNode` tree.

Expression Trees (expressions.hpp)

Predicate conditions are represented as binary trees of `Expression` nodes. Each node belongs to one of five categories:

Table 2: Expression node types

Type	Role	Example
COLUMN	Leaf — database column reference	<code>age</code> , <code>users.id</code>
CONSTANT	Leaf — literal value	<code>18</code> , <code>'admin'</code>
COMPARISON	Branch — relational operator	<code>age > 18</code>
LOGICAL_AND	Branch — conjunction	<code>A AND B</code>
LOGICAL_OR	Branch — disjunction	<code>A OR B</code>

Leaf constructors accept (`ExpressionType`, `value`); branch constructors accept (`ExpressionType`, `op`, `left`, `right`) with `unique_ptr` children. The recursive structure allows the semantic analyzer and optimizer to walk arbitrary WHERE clauses uniformly.

AST Plan Nodes (`ast_nodes.hpp`)

Each SQL clause maps to a typed `PlanNode` subclass. All nodes inherit the base class `PlanNode`, which holds the `NodeType` discriminant and a `vector<unique_ptr<PlanNode>>` child list.

Table 3: Plan node types and their SQL correspondences

Node Type	Kind	SQL Clause
<code>LogicalGetNode</code>	Logical	FROM / table scan
<code>LogicalFilterNode</code>	Logical	WHERE, HAVING
<code>LogicalProjectNode</code>	Logical	SELECT
<code>LogicalJoinNode</code>	Logical	JOIN (INNER / LEFT / CROSS)
<code>LogicalAggregateNode</code>	Logical	GROUP BY
<code>LogicalSortNode</code>	Logical	ORDER BY
<code>LogicalLimitNode</code>	Logical	LIMIT
<code>PhysicalHashJoinNode</code>	Physical	Hash Join algorithm
<code>PhysicalNestedLoopJoinNode</code>	Physical	Nested Loop Join algorithm

The critical design distinction is **logical vs. physical**: the parser always produces a purely logical tree. The CBO's sole job is to replace each `LogicalJoinNode` with the cheaper of `PhysicalHashJoinNode` or `PhysicalNestedLoopJoinNode`.

System Catalog (`catalog.hpp`)

The catalog module provides statistical metadata consumed by both the semantic analyzer (column existence) and the cost model (row and block counts). It is structured around a polymorphic interface:

```

1 class ICatalog {
2 public:
3     virtual TableStats getTableStats(const std::string& table_name)
4         const = 0;
5     virtual bool columnExists(const std::string& table_name,
6                               const std::string& column_name) const =
7         0;
8 };

```

MockCatalog. Reads table schemas and statistics from `src/catalog.json`. Stores data in `unordered_map` hash maps for $O(1)$ lookup. Supports runtime table injection via `addTable()`, enabling isolated unit tests without a live database.

PostgresCatalog. Implements the same interface against a live PostgreSQL instance using `popen` to invoke the `psql` binary as a hidden subprocess. Key design decisions:

- **$O(1)$ cardinality retrieval:** Queries `pg_class.reltuples` and `pg_class.relpages` after issuing `ANALYZE`, avoiding expensive `COUNT(*)` aggregates.
- **Low-level attribute validation:** Queries `pg_attribute` with `attnum > 0 AND NOT attisdropped` to correctly skip system columns and lazily-dropped attributes.
- **Memory safety:** Wraps the `FILE*` handle from `popen` in a `unique_ptr<FILE, decltype(&pclose)>`, guaranteeing no file-descriptor leak even if a query throws.

Semantic Analyzer (`semantic_analyzer.hpp`)

The semantic analyzer is a bottom-up tree walker that validates table existence, alias scope, and column references before any optimization is attempted.

Bottom-up traversal. The `visitNode()` function recurses on children *before* processing the current node. This guarantees that `LogicalGetNodes` (leaf tables) are registered into the `alias_to_table` map before any parent `LogicalFilterNode` or `LogicalJoinNode` references their columns.

Column validation. `validateColumn()` handles two cases:

- **Qualified names** (`orders.user_id`): splits on the dot, checks the alias is in scope, then asks the catalog whether the column exists in the resolved physical table.
- **Bare names** (`age`): iterates over all currently active tables and returns success if *any* table contains the column; throws `runtime_error` otherwise.

If the check fails, the engine prints a `Semantic Error:` message and stops before any optimizer is invoked.

Rule-Based Optimizer — Predicate Pushdown (`optimizer.hpp`)

The RBO implements **predicate pushdown**: relocating `LOGICAL_FILTER` nodes from above a `JOIN` to directly above the table they filter. This reduces the intermediate cardinality fed into the join by eliminating rows before the join executes.

3.6.1 AND-Predicate Splitting

A compound predicate `age > 18 AND amount > 500` cannot be pushed as a unit because it references columns from two different tables. The optimizer first decomposes it:

Algorithm: `splitAndPredicate`

Input: `FilterNode` with `LOGICAL_AND` predicate

Output: Two independent `FilterNodes` stacked on the child

- 1: **if** `predicate.type == LOGICAL_AND` **then**
- 2: $P_L \leftarrow \text{predicate.left}$
- 3: $P_R \leftarrow \text{predicate.right}$
- 4: $F_L \leftarrow \text{new FilterNode}(P_L)$
- 5: $F_L.\text{child} \leftarrow \text{original child}$
- 6: $F_R \leftarrow \text{new FilterNode}(P_R)$

```

7:    $F_R.\text{child} \leftarrow F_L$ 
8:   return PUSHDOWNFILTERS( $F_R$ )
9: end if

```

3.6.2 Single-Predicate Pushdown

After splitting, each individual filter is handled by the core pushdown logic:

Algorithm: pushDownFilter (single predicate)

Input: FilterNode F whose child is a JoinNode

Output: JoinNode with F inserted below it on the correct side

```

1:  $col \leftarrow \text{GETCOLUMNNAME}(F.\text{predicate})$ 
2:  $L \leftarrow \text{GETTABLENAME}(\text{Join.left})$ 
3:  $R \leftarrow \text{GETTABLENAME}(\text{Join.right})$ 
4: if catalog.columnExists( $L, col$ ) then
5:   Detach Join.left; attach it as  $F$ 's child
6:   Reattach  $F$  as Join.left
7: else if catalog.columnExists( $R, col$ ) then
8:   Detach Join.right; attach it as  $F$ 's child
9:   Reattach  $F$  as Join.right
10: else
11:   Leave  $F$  above Join (cross-table predicate)
12: end if
13: return Join

```

The net effect is illustrated for the query `WHERE age > 18 AND amount > 500`: both predicates are detached from above the JOIN and pushed to their respective leaf table scans, so the JOIN operates on the pre-filtered tuples only.

Cost Model (cost_model.hpp)

All cost estimates use **relative work units** calibrated to PostgreSQL's default cost constants. The anchor unit is one sequential block read.

Table 4: Cost constants and their rationale

Constant	Value	Rationale
SEQ_PAGE_COST	1.0	Baseline: sequential block read from disk
RANDOM_PAGE_COST	4.0	Random seek $\approx 4\times$ slower (HDD seek penalty)
CPU_TUPLE_COST	0.01	Fetching a row into CPU cache ($100\times$ faster than disk)
CPU_OPERATOR_COST	0.0025	Evaluating one predicate ($400\times$ faster than disk)
MEM_WRITE_COST	0.005	Writing one tuple into a hash table in RAM

3.7.1 Sequential Scan

$$C_{\text{seq}}(T) = B_T \cdot c_{\text{seq}} + N_T \cdot c_{\text{cpu-tuple}}$$

where B_T is the block count and N_T the tuple count of table T .

3.7.2 Nested Loop Join (NLJ)

$$C_{\text{NLJ}}(\text{outer}, \text{inner}) = \underbrace{B_O \cdot c_{\text{seq}} + N_O \cdot B_I \cdot c_{\text{seq}}}_{\text{I/O}} + \underbrace{N_O \cdot N_I \cdot c_{\text{op}}}_{\text{CPU}}$$

NLJ re-reads the entire inner table for each outer tuple, giving $O(N_O \cdot N_I)$ CPU cost — prohibitive for large tables.

3.7.3 Hash Join

$$C_{\text{HJ}}(\text{outer}, \text{inner}) = \underbrace{(B_O + B_I) \cdot c_{\text{seq}}}_{\text{I/O}} + \underbrace{N_I \cdot c_{\text{mem}}}_{\text{build}} + \underbrace{(N_O + N_I) \cdot c_{\text{cpu-tuple}} + N_O \cdot c_{\text{op}}}_{\text{probe}}$$

Both tables are read exactly once, giving $O(N_O + N_I)$ I/O — vastly superior for large inputs.

The `costJoin()` wrapper automatically places the *smaller* table as the inner (build) side of the Hash Join to minimize RAM consumption, then returns $\min(C_{\text{HJ}}, C_{\text{NLJ}})$.

Cost-Based Optimizer — DP Join Enumeration (`cost_based_optimizer.hpp`)

The CBO implements the System R dynamic programming algorithm extended to consider bushy trees. It operates in three phases.

3.8.1 Phase 1 — Tree Deconstruction

Before enumeration, the optimizer separates the plan tree into three parts:

- **Wrappers** (`collectWrappers`): PROJECT, AGGREGATE, SORT, LIMIT nodes above the join core. These are set aside and reattached after DP completes.
- **Tables** (`extractJoinTree`): Every LOGICAL_GET (with any pushed-down filter) is recorded with its `TableStats`.
- **Edges**: Each LOGICAL_JOIN becomes a `JoinEdge` recording the join type and ON condition between the subtree on its left and the subtree on its right.

3.8.2 Phase 2 — Bitmask DP Enumeration

Tables are indexed $0 \dots N - 1$. A subset of tables is represented as a bitmask (bit i set $\Leftrightarrow$ table i is in the subset). The memo table `memo[mask]` stores the cheapest plan and output cardinality for that subset.

Algorithm: DP Join Enumeration (Bushy)

Input: N tables with stats; join edges

Output: Optimal physical join plan for all N tables

- 1: **Base cases:** $\forall i \in [0, N)$: `memo[1  $\ll$  i]  $\leftarrow$  (SeqScanCost( $T_i$ ),  $T_i$ .stats, clone( $T_i$ .plan))`
- 2: **for** $size \leftarrow 2$ **to** N **do**
- 3: **for** each subset S with $|S| = size$ **do** $\triangleright$ via Gosper's Hack
- 4: **for** each non-empty proper subset $L \subset S$ **do**
- 5: $R \leftarrow S \setminus L$
- 6: **if** `memo[L]` and `memo[R]` exist **then**

```

7:      base ← cost(L) + cost(R)
8:      if base ≥ PRUNE_BUDGET then continue
9:      end if
10:     edge ← findEdge(L, R)
11:     (jt, cond) ← edge ≠ null ? (edge.type, edge.cond) : (CROSS, null)
12:     out ← estimateJoinOutput(stats(L), stats(R))
13:     plan ← chooseBestJoin(jt, cond, clone(L), clone(R))
14:     total ← base + plan.cost
15:     if total < cost(S) then
16:         memo[S] ← (total, out, plan)
17:     end if
18: end if
19: end for
20: end for
21: end for
22: return memo[(1 ≪ N) - 1].plan

```

Gosper’s Hack. Iterating all size- k subsets of $\{0, \dots, N-1\}$ in bitmask representation is done in $O(1)$ per step using:

$$c = S \& (-S), \quad r = S + c, \quad S_{\text{next}} = \left(\frac{(r \oplus S) \gg 2}{c} \right) | r$$

This avoids testing all 2^N masks to find those with exactly k bits.

Cardinality estimation. Join output size is estimated with a fixed selectivity $\sigma = 0.1$:

$$N_{\text{out}} = \max(1, N_L \cdot N_R \cdot \sigma), \quad B_{\text{out}} = \lceil N_{\text{out}} / \rho \rceil$$

where ρ is the higher of the two input tuple densities (tuples per block).

3.8.3 Phase 3 — Plan Reassembly

After `runDP()` completes, `memo[full_mask].plan` holds the optimal physical join core. `wrapNonJoinNodes()` reattaches the wrapper nodes collected in Phase 1, restoring the complete plan tree.

Physical join selection. `chooseBestJoin()` computes both Hash Join and NLJ costs via `CostModel` and returns a `PhysicalHashJoinNode` or `PhysicalNestedLoopJoinNode` — whichever is cheaper — as the winning node.

Summary of Results

Table 5: Test suite results

Test	Description	Key Feature Verified	Result
1	Simple scan	Parser, single table plan	PASS
2	Filter pushdown	RBO, filter preserved in CBO	PASS
3	3-table DP join	DP enumeration, optimal ordering	PASS
4	GROUP BY	AGGREGATE wrapper preservation	PASS
5	LEFT JOIN	Join type propagation	PASS
6	Bad table	Semantic error — table not found	PASS
7	Bad column	Semantic error — column not found	PASS
8	AND filter split	Dual predicate pushdown	PASS
9	4-table join aliases	Alias resolution, DP scaling	PASS

All nine tests pass. The engine correctly handles single-table queries, multi-table DP join reordering, predicate pushdown (including compound AND splitting), LEFT JOIN type preservation, AGGREGATE wrapper preservation, alias resolution, and semantic error gating.

In addition to the above test cases, two more test cases are added, we made a separate report to describe few important test cases with images, please go through it as well.

Web Dashboard

The web interface exposes the optimizer as a microservice using a **decoupled three-layer architecture**:

1. **Flask (app.py)**: Receives SQL from the browser as JSON, writes it to a temporary file, invokes `./optimizer_test temp_query.sql --html-out` as a subprocess, and returns the generated `result.json` to the client.
2. **Bootstrap 5 (dashboard.html)**: Provides a responsive layout with a SQL editor, cost badges, and a three-tab panel (Logical $\rightarrow$ RBO $\rightarrow$ CBO).
3. **D3.js**: Parses the JSON plan trees using `d3.hierarchy()` and `d3.tree()`, computes node coordinates, and renders color-coded SVG trees (yellow for SCAN, blue for JOIN, red for FILTER, green for PROJECT) with pan/zoom and hover tooltips.

The C++ binary is completely independent of the web layer; it operates correctly as a standalone CLI tool. The Flask server simply wraps it, making the system highly modular and testable.